

Práctica 2. RL tabular sin modelo

(Duración: 2 sesiones)

PREPARACIÓN DEL ENTORNO DE PROGRAMACIÓN

La práctica se ejecuta con una instalación local de Python. No hace falta Docker, ni WSL, ni ningún servidor X11. Si ya has hecho la Práctica 1, te sirve el mismo entorno: solo hay que añadir `matplotlib`.

1. **Instalar Python, de la versión 3.10 a la 3.13.** Si no lo tienes, descárgalo de la [página oficial de Python](#) o utiliza una instalación de Anaconda. La 3.14 todavía no sirve, porque `pygame` no tiene versión para ella.
2. **Crear un entorno virtual** para la asignatura y activarlo. Desde la carpeta de la práctica:

```
python -m venv .venv
.venv\Scripts\activate      (Windows)
source .venv/bin/activate   (macOS y Linux)
```

3. **Instalar las dependencias** en el entorno virtual que uses:

```
pip install numpy gymnasium pygame matplotlib
```

4. **Comprobar que todo funciona:** con el entorno virtual activado, ejecuta la siguiente instrucción desde el terminal.

```
python main.py --algorithm sarsa --episodes 100
```

Debe imprimir la configuración del entrenamiento y terminar con este mensaje, que avisa de que Sarsa todavía no está implementado:

```
NotImplementedError: Completa episodio_sarsa.
```

Si en su lugar aparece `ModuleNotFoundError`, falta el paquete que nombra el error: instálalo con `pip install` en el mismo entorno y vuelve a ejecutar la instrucción hasta obtener ese mensaje.

Cualquier editor sirve para trabajar. Si usas VSCode, basta con instalar la extensión de Python y seleccionar el intérprete del entorno virtual.

1. Evaluación de la práctica

La segunda práctica del laboratorio de aprendizaje por refuerzo se divide en dos partes. La primera consiste en un trabajo previo individual que cada alumno debe subir a la entrega que habrá habilitada en Moodle antes del inicio de la sesión presencial del laboratorio.

La segunda parte consiste en la implementación y el análisis de los algoritmos Sarsa y Q-learning en un entorno tabular. El entregable correspondiente a esta parte de la práctica consiste en un informe en el que se responderán las preguntas que haya a lo largo del enunciado y en el que se deberá mostrar que se han comprendido los aspectos más importantes del aprendizaje de los agentes. Por ello se valorará que las respuestas estén debidamente justificadas.

El plazo para entregar el informe es de una semana desde la realización de la sesión de laboratorio.

La evaluación final de la práctica ponderará tanto el trabajo previo como el informe, siendo obligatorio haber entregado el trabajo previo antes del laboratorio para que se tenga en cuenta la nota del informe.

2. Descripción del problema

Se usa el mismo entorno que en la Práctica 1, pero ahora el agente **no dispone del modelo de transiciones**: tiene que aprender interactuando con él. Las recompensas r_s , r_{obs} y r_{goal} ya no están fijadas en el enunciado, sino que las elige el alumno. La Figura 1 recuerda la configuración del entorno.

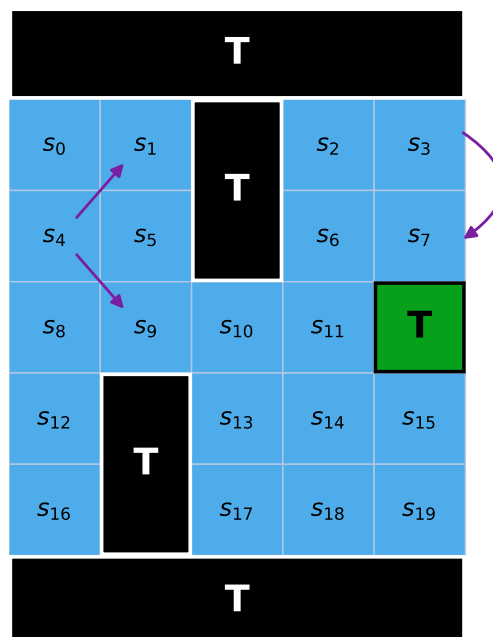


Figura 1. Entorno de la práctica. Solo llevan número las celdas en las que el agente puede encontrarse: son los 20 estados no terminales, s_0 a s_{19} . Todas las celdas marcadas con T son el mismo estado terminal, al que se llega tanto al chocar con un obstáculo (celdas negras) como al alcanzar el objetivo (celda verde). Las flechas ilustran las dos acciones desde s_4 y la caída del borde derecho.

ENTORNO

- Las celdas forman una matriz 7x5. Las celdas azules son los estados en los que el jugador puede encontrarse y se numeran de izquierda a derecha y de arriba a abajo, de s_0 a s_{19} (ver Figura 1).
- Si el agente se mueve desde uno de estos estados a otro estado no terminal, obtiene una recompensa r_s .
- Las celdas negras son obstáculos que el agente ha de evitar, ya que si cae en uno, se recibe una recompensa r_{obs} y se termina el episodio.
- La celda verde representa el objetivo al que hay que llegar, el cual implica una recompensa r_{goal} y también termina el episodio.

- Las celdas negras y la celda verde **no son estados distintos entre sí**: todas ellas son el mismo estado terminal T , en el que el episodio acaba. Lo que las diferencia es la recompensa de la transición que lleva hasta él, no el estado de destino. T es absorbente.

MOVIMIENTOS DEL JUGADOR

- El jugador tiene dos posibles acciones: arriba-derecha (del estado s_4 se mueve al s_1) o abajo-derecha (del estado s_4 se mueve al s_9).
- El entorno tiene bordes sólo por la derecha. Si desde un estado de la última columna se realiza cualquiera de las dos acciones, se cae en la misma columna pero una celda por debajo (del estado s_3 se pasa al s_7 se tome la acción que se tome). Ver Figura 1.
- Si no se indica lo contrario, el entorno es determinista.

ASUNCIONES (si no se especifica otra cosa)

- Factor de descuento $\gamma = 0,9$
- $\epsilon_{entrenamiento} = 0,2$ (es un valor sugerido, pero puede cambiarse)
- En evaluación se sigue la política greedy ($\epsilon = 0$)
- Número de episodios de entrenamiento, a fijar por el alumno
- $Q_0(s, a)$, a fijar por el alumno
- r_s, r_{obs} y r_{goal} , a fijar por el alumno
- $s_{init} = \{s_0, s_4, s_8, s_{12}\}$: el entorno sortea el estado inicial de cada episodio entre esos cuatro
- T : único estado terminal, absorbente

3. Trabajo previo (2 ptos)

Calcular los valores óptimos $q_*(s, a)$ de todos los pares estado-acción del entorno, es decir, de los 20 estados no terminales (s_0 a s_{19}) con sus dos acciones, a partir de los resultados de programación dinámica (Policy Iteration y Value Iteration) obtenidos en la Práctica 1. Se toman los mismos valores con los que se resolvió allí: entorno determinista, $\gamma = 0,9$, $r_s = -1$, $r_{obs} = -5$ y $r_{goal} = +5$. Presentar los resultados en una tabla.

4. Sarsa (4 ptos)

A continuación se debe implementar y analizar el algoritmo Sarsa. Para ello se dispone de una carpeta en la que se encontrarán los siguientes archivos:

- `main.py`: es el script principal. Entrena, imprime la tabla Q aprendida y la política greedy, y dibuja las gráficas del informe. Todos los hiperparámetros se pasan por la línea de comandos, así que **no hay que editarlo**:

```
python main.py --algorithm <sarsa/q_learning/both>
--episodes 20000 --alpha 0.5 --q_init 5 --graficas
```

`python main.py --help` lista el resto de opciones (γ , ϵ , las tres recompensas, la semilla y los estados cuya Q se registra).

- `env.py`: contiene la definición del entorno donde el agente interactúa. Es el mismo fichero de la Práctica 1. Proporciona el número de estados nS (21: los 20 visitables y el terminal T) y el número de acciones nA . No es necesario modificar nada dentro de este código durante la práctica. **El modelo de transiciones no se puede usar**: los algoritmos de esta práctica aprenden solo de la experiencia.
- `entrenamiento.py`: repite episodios y registra los datos que necesitan las gráficas (retorno, pasos, episodios acabados en el objetivo y evolución de Q). Tampoco hay que modificarlo.
- `sarsa.py`: implementación base para el algoritmo Sarsa. Dentro se encontrarán un método para la política ϵ -greedy y otro para ejecutar un episodio.
- `q_learning.py`: implementación base para el algoritmo Q-learning, que se usa en el apartado siguiente.

Programar y entrenar dentro del espacio correspondiente (marcado con START CODE HERE ... END CODE HERE) el algoritmo de aprendizaje Sarsa. No se debe modificar nada que no esté dentro de este espacio, los argumentos que reciben los métodos son suficientes para desarrollar el algoritmo.

Analizar cómo converge $Q(s, a)$ para s_6 y para s_{13} , y cómo evoluciona para distintos valores de los hiperparámetros (número de episodios, α , ϵ y Q_0).

5. Q-learning (4 ptos)

Programar y entrenar dentro del espacio correspondiente (marcado con START CODE HERE ... END CODE HERE) en `q_learning.py` el algoritmo de aprendizaje Q-learning. Analizar cómo converge $Q(s, a)$ para s_6 y para s_{13} , y su evolución para distintos valores de los hiperparámetros.

Comparar por último el aprendizaje de los dos algoritmos: ¿llegan a la misma política?, ¿llegan a los mismos valores de Q ?, ¿cuál de los dos obtiene mejor retorno **durante** el entrenamiento, y por qué?